

Parmi les quatre propositions suivantes, écrire sur la copie l'instruction à placer à la ligne 4 du code de la fonction `fusion` :

Proposition 1	<code>while i1 < len(liste1) and i2 < len(liste2):</code>
Proposition 2	<code>while i1 < len(liste1) or i2 < len(liste2):</code>
Proposition 3	<code>while i1 + i2 < len(liste1) + len(liste2):</code>
Proposition 4	<code>while liste1[i1 + 1] < liste2[i2]):</code>

- c. Recopier et compléter sur la copie la fonction récursive `tri_fusion` suivante, qui prend en paramètre une liste non triée et la renvoie sous la forme d'une nouvelle liste triée.

On utilisera la fonction `fusion` de la question précédente.

Exemple : `tri_fusion([140, 115, 78, 94])` doit renvoyer
`[78, 94, 115, 140]`

Aide Python : si `L` est une liste, l'instruction `L[i:j]` renvoie la liste constituée des éléments de `L` indexés de `i` à `j-1`.

Par exemple, si `L=[5, 4, 8, 7, 3]`, `L[2:4]` vaut `[8, 7]`.

Numéro de lignes	Fonction <code>tri_fusion</code>
1	<code>def tri_fusion(liste):</code>
2	<code> if len(liste) <= 1:</code>
3	<code> return liste</code>
4	<code> à compléter</code>

Exercice 3 (5 Points)

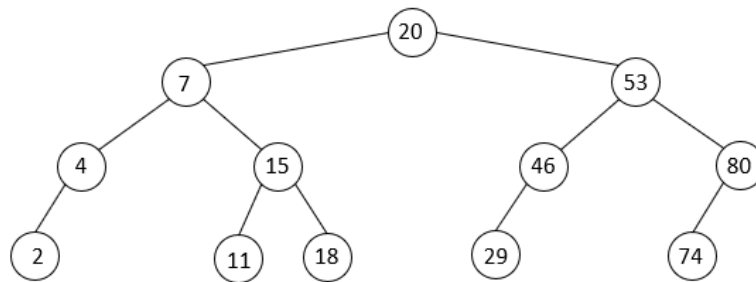
Cet exercice traite des arbres et de l'algorithmique.

Dans cet exercice, la taille d'un arbre est égale au nombre de ses nœuds et on convient que la hauteur d'un arbre ne contenant qu'un nœud vaut 1.

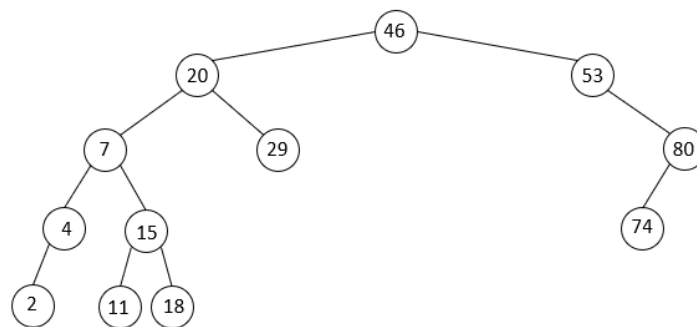
On utilisera la définition suivante : un arbre binaire de recherche est un arbre binaire, dans lequel

- on peut comparer les valeurs des nœuds : ce sont par exemple des nombres entiers, ou des lettres de l'alphabet ;
- si `x` est un nœud de cet arbre et `y` est un nœud du sous-arbre gauche de `x`, alors il faut que `y.valeur < x.valeur`
- si `x` est un nœud de cet arbre et `y` est un nœud du sous-arbre droit de `x`, alors il faut que `y.valeur ≥ x.valeur`

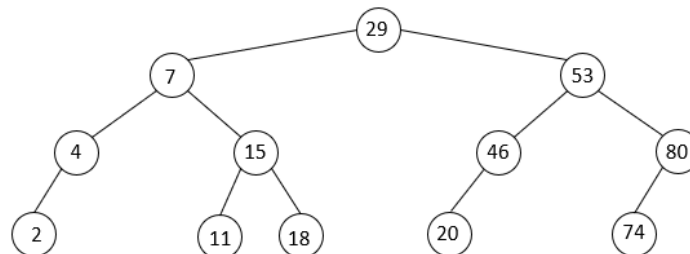
1. Parmi les trois arbres dessinés ci-dessous, recopier sur la copie le numéro correspondant à celui qui n'est pas un arbre binaire de recherche. Justifier.



Arbre 1



Arbre 2



Arbre 3

Une classe `ABR`, qui implémente une structure d'arbre binaire de recherche, possède l'interface suivante :

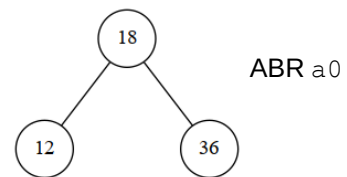
Numéro de lignes	Classe <code>ABR</code>
1	<code>class ABR :</code>
2	<code> def __init__(self, valeur, sa_gauche, sa_droit):</code>
3	<code> self.valeur = valeur #valeur de la racine</code>
4	<code> self.sa_gauche = sa_gauche #sous-arbre gauche</code>
5	<code> self.sa_droit = sa_droit #sous-arbre droit</code>
6	<code> def inserer_noeud(self, valeur):</code>
7	<code> """Renvoie un nouvel ABR avec le nœud de valeur 'valeur'</code>
8	<code> inséré comme nouvelle feuille à sa position correcte"""</code>
9	<code> # code non étudié dans cet exercice</code>
...	

On prendra la valeur `None` pour représenter un sous-arbre vide.

2. La construction d'un ABR se fait en insérant progressivement les valeurs à partir de la racine : la méthode `insérer_noeud` (dont le code n'est pas étudié dans cet exercice) place ainsi un nœud à sa "bonne place" comme feuille dans la structure, sans modifier le reste de la structure. On admet que la position de cette feuille est unique.

a. En utilisant les méthodes de la classe ABR :

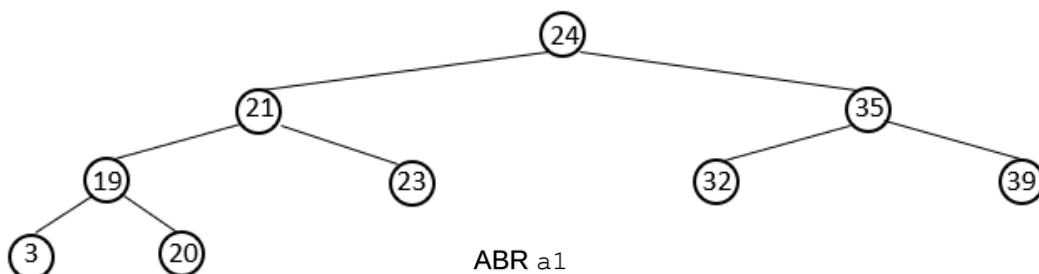
- écrire l'instruction Python qui permet d'instancier un objet `a0`, de type ABR, ayant un seul nœud (la racine) de valeur 18.
- écrire une séquence d'instructions qui permet ensuite d'insérer dans l'objet `a0` les deux feuilles de l'arbre de valeurs 12 et 36.



Selon l'ordre dans lequel les valeurs sont insérées, on construit des ABR ayant des structures différentes.

Voilà par exemple ci-dessous un ABR (nommé `a1`) obtenu en créant une instance de type ABR ayant un seul nœud (la racine) de valeur 24 puis en insérant successivement les valeurs dans l'ordre suivant :

21 ; 35 ; 19 ; 23 ; 32 ; 39 ; 3 ; 20



- b. Dessiner sur la copie l'ABR (nommé `a2`) que l'on obtiendrait en créant une instance de type ABR ayant un seul nœud (la racine) de valeur 3 puis en insérant successivement les valeurs dans l'ordre suivant :

20 ; 19 ; 21 ; 23 ; 32 ; 24 ; 35 ; 39

- c. Donner la hauteur des ABR `a1` et `a2`.

- d. On complète la classe ABR avec une méthode `calculer_hauteur` qui renvoie la hauteur de l'arbre.

Recopier sur la copie les lignes 10 et 13 en les complétant par des commentaires et la ligne 14 en la complétant par une instruction dans le code ci-après de cette méthode.

On pourra utiliser la fonction Python `max` qui prend en paramètres deux nombres et renvoie le maximum de ces deux nombres.

Numéro de lignes	Méthode calculer_hauteur
1	def calculer_hauteur(self):
2	""" Renvoie la hauteur de l'arbre"""
3	if self.sa_droit is None and self.sa_gauche is None:
4	#l'arbre est réduit à une feuille
5	return 1
6	elif self.sa_droit is None
7	#arbre avec une racine et seulement un sous-arbre gauche
8	return 1 + self.sa_gauche.calculer_hauteur()
9	elif self.sa_gauche is None:
10	# à compléter
11	return 1 + self.sa_droit.calculer_hauteur()
12	else:
13	# à compléter
14	return à compléter

3. La différence de hauteur entre l'ABR a1 et l'ABR a2 aura des conséquences lors de la recherche d'une valeur dans l'ABR.

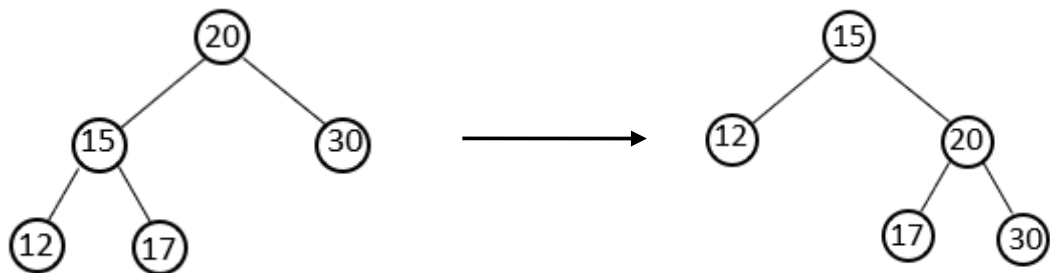
a. Recopier et compléter sur la copie les lignes 6, 8, 11 et 13 du code ci-dessous de la méthode `rechercher_valeur`, qui permet de tester la présence ou l'absence d'une valeur donnée dans l'ABR :

Numéro de lignes	Méthode rechercher_valeur
1	def rechercher_valeur(self, v):
2	"""
3	Renvoie True si la valeur v est trouvée dans l'ABR,
4	False sinon
5	"""
6	if à compléter
7	return True
8	elif à compléter and self.sa_gauche is not None:
9	return self.sa_gauche.rechercher_valeur(v)
10	elif v > self.valeur and self.sa_droit is not None:
11	return à compléter
12	else:
13	return à compléter

b. On admet que le nombre de fois où la méthode `rechercher_valeur` est appelée pour rechercher la valeur 39 dans l'ABR a2 est 7.
Donner le nombre de fois où la méthode `rechercher_valeur` est appelée pour rechercher la valeur 20 dans l'ABR a1.

4. Il existe des algorithmes pour modifier la structure d'un ABR, afin par exemple de diminuer la hauteur d'un ABR ; on s'intéresse aux algorithmes appelés *rotation*, consistant à faire "pivoter" une partie de l'arbre autour d'un de ses nœuds.

L'exemple ci-dessous permet d'expliquer l'algorithme pour réaliser une rotation droite d'un ABR autour de sa racine :



On appelle <i>pivot</i> le sous-arbre gauche de la racine de l'arbre	
Le sous-arbre droit du pivot devient le sous-arbre gauche de la racine	
La racine ainsi modifiée devient le sous-arbre droit du pivot et la racine du pivot devient la nouvelle racine de l'ABR	

On admet que ces transformations conservent la propriété d'ABR de l'arbre.

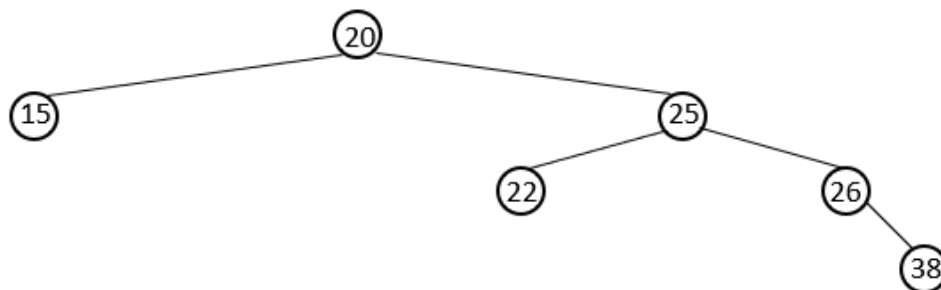
La méthode `rotation_droite` ci-après renvoie une nouvelle instance de type ABR, correspondant à une rotation droite de l'objet de type ABR à partir duquel elle est appelée :

Numéro de lignes	Méthode <code>rotation_droite</code>
1	<code>def rotation_droite(self):</code>
2	<code> """ Renvoie une instance d'un ABR apres une rotation droite</code>
3	<code> On suppose qu'il existe un sous-arbre gauche"""</code>
4	<code> pivot = self.sa_gauche</code>
5	<code> self.sa_gauche = pivot.sa_droit</code>
6	<code> pivot.sa_droit = self</code>
7	<code> return ABR(pivot.valeur,pivot.sa_gauche,pivot.sa_droit)</code>

Pour réaliser une rotation gauche, on suivra alors l'algorithme suivant :

- on appelle *pivot* le sous-arbre droit de la racine de l'arbre,
- le sous-arbre gauche du pivot devient le sous-arbre droit de la racine,
- la racine ainsi modifiée devient le sous-arbre gauche du pivot et la racine du pivot devient la nouvelle racine de l'ABR

- a. En suivant les différentes étapes de cet algorithme, dessiner l'arbre obtenu après une rotation gauche de l'ABR suivant :



- b. Écrire le code d'une méthode Python `rotation_gauche` qui réalise la rotation gauche d'un ABR autour de sa racine.